Insecure API Endpoint Exploitation response

Web / API · **Critical** severity · SOC IR Playbook

Incident type	Application-Layer Exploit - API Abuse
Severity	Critical
Priority	High
Detection sources	SIEM, API Gateway Logs, WAF, Runtime Application Security Protection (RASP), Application Logs, Threat Intelligence Feeds

Scenario

An attacker discovers and exploits insecure API endpoints-such as those lacking authentication, rate limiting or proper input validation-to perform unauthorised data access, modify business logic, escalate privileges or carry out denial-of-service (DoS) attacks.

MITRE ATT&CK

T1001.003, T1190, T1499, T1539

Preparation

Enable API gateway logging Track request methods, source IPs, endpoints and parameters Enforce input validation Implement strict validation and sanitisation in backend logic Deploy rate limiting & throttling Prevent abuse through bulk or automated requests Require authentication & authorisation Use OAuth, JWT, API keys with role enforcement Monitor API behaviour Use anomaly detection to flag unexpected access patterns

Detection & Analysis

Receive alert Excessive API calls, unauthorised data access, error spikes (e.g., 403s, 500s) Identify affected endpoints Analyse logs to determine what APIs were accessed and how Review query patterns Look for signs of enumeration, injection, mass scraping or business logic abuse Correlate with user/IP Determine whether the actor is internal, authenticated or abusing open APIs

Containment

Block offending IPs or tokens At the API gateway, WAF or CDN level Disable vulnerable endpoint Temporarily disable or restrict access to the affected API Throttle suspicious traffic Enforce tighter rate limits for abusive patterns Alert development and product teams To assist with containment and business risk assessment

Eradication

Fix insecure API logic Add authentication, access control and input validation Patch or redeploy backend service If vulnerability is rooted in code or library Rotate affected credentials or API keys Especially if token theft or privilege abuse occurred Remove injected data If attacker used the API to insert malicious or corrupt data

Recovery

Restore secure access After verifying fix, monitor closely for any signs of bypass or regression Inform affected users If personal or sensitive data was accessed or altered Retest affected APIs Conduct regression and security testing before full reactivation Resume full service Once security and stability are verified in production environments

Lessons Learned & Reporting

Conduct a root cause analysis Identify design, configuration or development oversight Update SDLC policies Include security testing for API endpoints (e.g., OWASP API Top 10) Enhance monitoring and alerting Add detection for enumeration, excessive calls or unusual inputs Train developers On secure API design, proper authentication and error handling Document the incident Include timeline, attacker behaviour, impact and mitigations applied

Tools typically involved

API Gateways (e.g., Kong, AWS API Gateway, Apigee, Azure API Management); WAF (e.g., Cloudflare, AWS WAF, Imperva); SIEM (e.g., Sentinel, Splunk); RASP and Runtime Protection (e.g., Signal Sciences, Contrast Security); Application performance/logging tools (e.g., Datadog, New Relic); DAST tools (e.g., Burp Suite, OWASP ZAP)

Success metrics (SLA targets)

Detection Time <10 minutes from abnormal activity Endpoint Restriction Time <30 minutes after confirmation Patch/Code Fix Deployment <24-48 hours for critical API bugs Retest & Recovery Time Within 72 hours Developer Training Coverage 100% of backend/API teams briefed within 7 days